Student Enrollment & Course Management System (SECMS)

A Salesforce-Based Application for Digitizing Agricultural Operations

Project Documentation

Annamalai University

Arignar Anna Government Arts College,Villupuram

Team ID: 6ab21bbf11b716a3e52c59a2

Team Leader	**VETRIVEL S**
Team Member	GOBINATH M
Team Member	ABISHEK V
Team Member	SEENUVASAN V

Table of Contents

1. Introduction
2. Ideation Phase
3. Requirement Analysis Phase
4. Project Design Phase
5. Project Planning & Scheduling
6. Project Development Phase (Milestones 1–13)
7. Output Screenshots — Object Records
8. Output Screenshots — Reports & Dashboard
9. Functional & Performance Testing
10. Advantages & Disadvantages
11. Conclusion
12. Future Scope
13. Appendix

1. Introduction

1.1 Project Overview

The Student Enrollment & Course Management System (SECMS) is a comprehensive Salesforce application designed to streamline and automate the academic enrollment operations of an educational institute. This system enables administrators, enrollment officers, and instructors to efficiently manage student records, course offerings, enrollments, instructor assignments, approval processes, payments, and reporting — all within a unified Salesforce environment.

This project aims to centralize all student, course, and enrollment-related data within a single Salesforce application to ensure easy data access, structured workflows, and accurate reporting. The system allows Enrollment Officers to manage student admissions, course registrations, and communication effectively, while instructors and administrators gain full transparency into course allocations and academic progress. The solution also enables automated instructor assignment, task creation, validation checks, and multi-level approval processes for enrollment management.

With Salesforce Flows, Approval Processes, Validation Rules, Reports, and Dashboards, the organization can ensure that all enrollment operations — from admission to course completion — are handled systematically and with continuous monitoring of key performance indicators such as enrollment volume, revenue, student status distribution, and pending approvals.

1.2 Purpose

The primary purpose of SECMS is to replace manual and fragmented enrollment processes with a scalable, automated, and secure Salesforce solution that ensures:

- Accurate student data management
- Structured enrollment workflows
- Automated approvals and validations
- Transparent reporting and monitoring

2. Ideation Phase

2.1 Problem Statement

PS	I am (Customer)	I'm trying to	But	Because	Which makes me
PS-1	Enrollment Officer	manage student enroll	ents efficient	ssy as they are manual and	unintegrated, erro -prone
PS-2	Administrator	track enrollments and r	reporting is delayed	data is not centralized	uncertain in decisi n-making

2.2 Empathy Map Canvas — User Persona: Enrollment Officer

Dimension	Details
Think & Feel	Concerned about double bookings and revenue loss
Hear	Complaints from students; feedback from management
See	Multiple spreadsheets and overflowing emails
Say & Do	Rechecks data, follows up manually
Pain	Duplicate entries, approval bottlenecks
Gain	Automation, dashboards, faster processing

2.3 Brainstorming & Idea Prioritization

Step 1 – Brainstormed Ideas

- Spreadsheet automation
- Standalone enrollment portal
- ERP integration
- Salesforce-based enrollment system

Step 2 – Grouping

- Manual solutions — Spreadsheet automation
- Semi-automated solutions — Standalone enrollment portal, ERP integration
- Fully automated cloud solutions — Salesforce-based enrollment system

Step 3 – Prioritization

The Salesforce-based SECMS was selected due to **high feasibility** and **high impact**, positioning it in the top-right quadrant of the Importance vs. Feasibility matrix.

3. Requirement Analysis Phase

3.1 Customer Journey Map

The journey map traces the enrollment lifecycle across seven stages:

- Awareness & Registration
- Course Selection
- Enrollment Request
- Fee Payment
- Approval
- Instructor Assignment
- Reporting & Completion

3.2 Solution Requirements — Functional Requirements

FR No	Epic	Description
FR-1	Student Management	Manage student records
FR-2	Course Management	Create and manage courses
FR-3	Enrollment Processing	Request, approve, reject enrollments
FR-4	Instructor Assignment	Auto-assign instructors
FR-5	Reporting	Dashboards and reports

Non-Functional Requirements

NFR-1: Usability

The system shall provide a simple, intuitive, user-friendly interface enabling all user roles to perform operations efficiently with minimal training, via clearly labeled tabs, dashboards, and forms.

- Quick student registration form
- One-click enrollment approval
- Role-based dashboards

NFR-2: Security

The system shall ensure confidentiality, integrity, and protection of student and institutional data through secure authentication, authorization, and access controls.

- Only admins can modify course details
- Instructors view only assigned students
- Students cannot access other student records

NFR-3: Reliability

The system shall operate consistently and accurately without failures during enrollment processing, approvals, and reporting.

- No duplicate course registration

- Accurate status updates after approvals
- Consistent report generation

NFR-4: Performance

The system shall provide fast response times for all operations, especially during peak admission periods.

- Quick dashboard refresh
- Fast search for student records
- No lag during bulk uploads

NFR-5: Scalability

The system shall support future growth in students, courses, departments and users without degradation in performance.

- Add new departments without redesign
- Handle increased admissions annually
- Extend to mobile or portal integration

NFR-6: Availability

The system shall be continuously accessible during institutional working hours with minimal downtime.

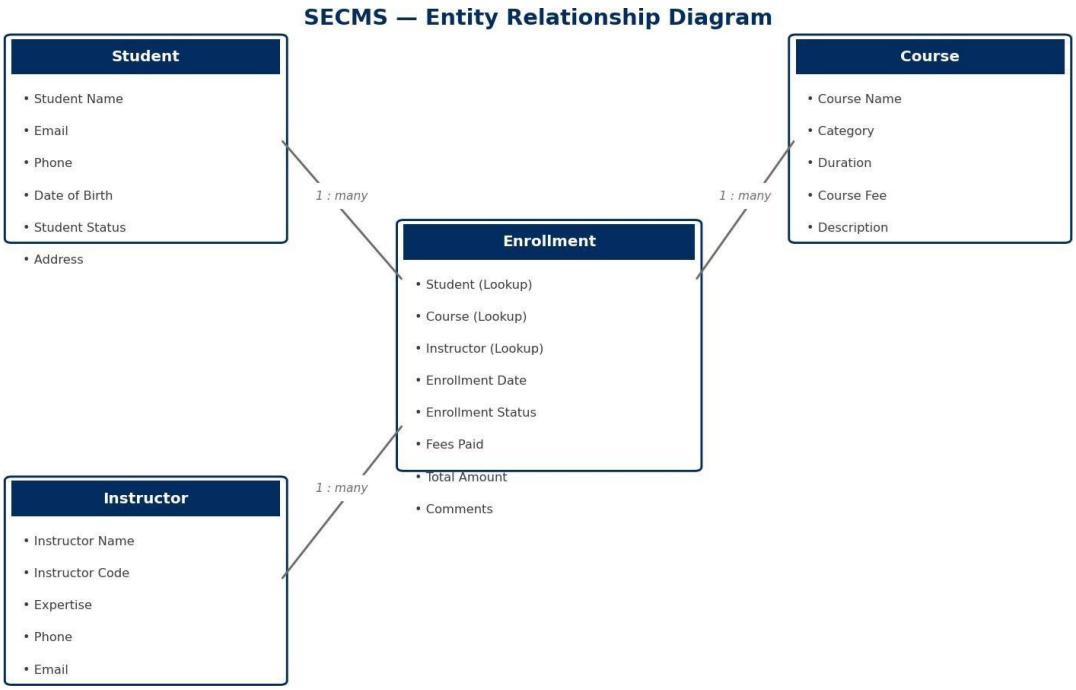
- 24/7 cloud access
- Reliable during peak admission season
- Automatic backup of student records

3.3 Data Flow Diagram

User inputs student and enrollment data → System validates and stores records → Approval process routes data → Reports and dashboards consume processed data. The system processes enrollment requests, validates data, routes approvals, assigns instructors, and generates reports.

Entity Relationship Diagram

The data model centres on the Enrollment object, which links a Student, a Course, and an Instructor through three lookup relationships, each supporting a one-to-many cardinality from the parent object to Enrollment.



3.4 Technology Stack

Layer	Technology
UI	Salesforce Lightning
Logic	Salesforce Flows
Database	Salesforce Objects
Automation	Validation Rules, Approval Process
Reporting	Reports & Dashboards

4. Project Design Phase

4.1 Problem–Solution Fit

SECMS directly addresses fragmented enrollment workflows — fragmented spreadsheets, manual paperwork, delayed approvals, and scattered systems — by aligning user behavior, constraints, and automation using Salesforce-native capabilities: unified dashboards, automated approvals, a centralized database, and satisfied administrators.

4.2 Proposed Solution

Parameter	Description
Problem	Manual enrollment handling
Solution	Salesforce-based SECMS
Novelty	End-to-end automation
Social Impact	Improved student experience
Revenue Model	Institutional deployment
Scalability	Multi-department support

4.3 Solution Architecture

The architecture integrates four core Salesforce objects — Student, Course, Enrollment, and Instructor — feeding into four automation layers: Automation Flows, Approval Processes, Dashboards, and Security, all running on the Salesforce Platform.

5. Project Planning & Scheduling

5.1 Project Planning

- Agile methodology
- Epics → Stories → Story Points
- Sprint-based execution
- Velocity calculation used for estimation

Product Backlog & Sprint Schedule

Sprint	Epic	USN	User Story / Task	Pts	Priority
Sprint-1	Developer Setu	USN-1	Create Salesforce developer account	1	High
Sprint-1	Data Modeling	USN-2	Create custom objects (Student, Course, Enrollme n3t, Inst uHctiogrh)		
Sprint-2	Data Modeling	USN-3	Dedicated tabs for each object	3	High
Sprint-2	Data Modeling	USN-4	Create fields for student, course, enrollment data	5	High
Sprint-2	Data Modeling	USN-5	Centralized Lightning App	3	High
Sprint-2	Data Modeling	USN-6	Define relationships between objects	5	Medium
Sprint-2	Data Modeling	USN-7	Customized page layouts	5	High
Sprint-3	Automation	USN-8	Validate mandatory fields and enrollment limits	3	High
Sprint-3	Automation	USN-9	Automated flows for enrollment & notifications	3	High
Sprint-3	Automation	USN-10	Approval process for enrollment requests	5	High
Sprint-4	Security	USN-11	Role-based security configuration	5	High
Sprint-5	Reports	USN-12	Reports on enrollments, courses, instructors	4	High
Sprint-6	Dashboards	USN-13	Visual dashboards for enrollment statistics	4	Medium

Sprint Tracker, Velocity & Burndown

Sprint	Points	Duration	Start	Planned End	Completed	Actual Release
Sprint-1	20	6 Days	24 Oct 2025	29 Oct 2025	20	29 Oct 2025
Sprint-2	20	6 Days	31 Oct 2025	05 Nov 2025	18	06 Nov 2025
Sprint-3	20	6 Days	07 Nov 2025	12 Nov 2025	20	12 Nov 2025
Sprint-4	20	6 Days	14 Nov 2025	19 Nov 2025	20	19 Nov 2025
Sprint-5	10	4 Days	21 Nov 2025	24 Nov 2025	10	24 Nov 2025
Sprint-6	10	4 Days	26 Nov 2025	29 Nov 2025	10	29 Nov 2025

6. Project Development Phase

The build follows fourteen milestones executed on a Salesforce Developer Edition org, taking the system from account setup through to a fully secured, reportable application.

- Milestone 1 — Salesforce Developer Account Creation
- Milestone 2 — Custom Object Creation (Student, Course, Enrollment, Instructor)
- Milestone 3 — Tabs Creation
- Milestone 4 — Lightning App Creation
- Milestone 5 — Fields Creation
- Milestone 6 — Create Relationships
- Milestone 7 — Page Layout Customization
- Milestone 8 — Validation Rules
- Milestone 9 — Flows (Automation)
- Milestone 10 — Approval Process
- Milestone 11 — Reports
- Milestone 12 — Dashboards
- Milestone 13 — Security Setup
- Milestone 14 — Conclusion

What You'll Learn

- Real-time Salesforce project implementation
- Designing custom objects and relationships
- Creating page layouts & record types
- Writing validation rules
- Building automation with flows
- Creating approval workflows
- Developing reports & dashboards
- Setting up security with profiles, permission sets, and sharing
- End-to-end application testing

Milestone 1 — Salesforce Developer Account Setup

Activity 1: Creating Developer Account

A free Salesforce Developer Edition org was created at developer.salesforce.com/signup with the following details entered on the sign-up form:

- First Name & Last Name
- Email address
- Role: Developer
- Company: College Name
- Country: India
- Postal Code: institution PIN code
- Username: a combination of name and organization, e.g. `username@organization.com` — this need not be a real mailbox

After filling in these fields, **Sign Me Up** was clicked to submit the registration.

Activity 2: Account Activation

- Opened the inbox of the email used at sign-up and located the "Welcome to Salesforce: Verify your account" message (arrives within 5–10 minutes)
- Clicked **Verify Account**
- Set a new password meeting the minimum policy (8+ characters, 1 letter, 1 number), answered a security question, and clicked **Change Password**
- Was redirected automatically to the Salesforce Setup Home page, confirming the org was ready for configuration

Milestone 2 — Custom Object Creation

Activity 1: Creating the Student Object

Purpose: to store and manage information about students such as personal details, contact information, and enrollment status.

- Navigated to Setup → Object Manager → Create → Custom Object
- Label: **Student**; Plural Label: **Students**; Object Name: Student
- Record Name: Student Name; Data Type: Text
- Under Optional Features: enabled Allow Reports, Allow Activities, Track Field History
- Under Search Status: enabled Allow Search
- Deployment Status set to Deployed; clicked **Save**

The same procedure was repeated to create the **Course**, **Instructor**, and **Enrollment** custom objects, giving the application its four foundational data tables.

Milestone 3 — Fields

Student Object Fields

Field Label	Data Type
Student Name	Text (Standard)
Email	Email
Field Label	**Data Type**
Phone	Phone

Date of Birth	Date
Student Status	Picklist
Address	Text Area

Course Object Fields

Field Label	Data Type
Course Name	Text (Standard)
Category	Picklist
Duration	Number/Text
Course Fee	Currency
Description	Text Area

Instructor Object Fields

Field Label	Data Type
Instructor Name	Text (Standard)
Instructor Code	Text
Expertise	Picklist
Phone	Phone
Email	Email

Enrollment Object Fields

Field Label	Data Type
Student	Lookup (Student)
Course	Lookup (Course)
Instructor	Lookup (Instructor)
Enrollment Date	Date
Enrollment Status	Picklist
Fees Paid	Checkbox
Total Amount	Currency
Comments	Text Area

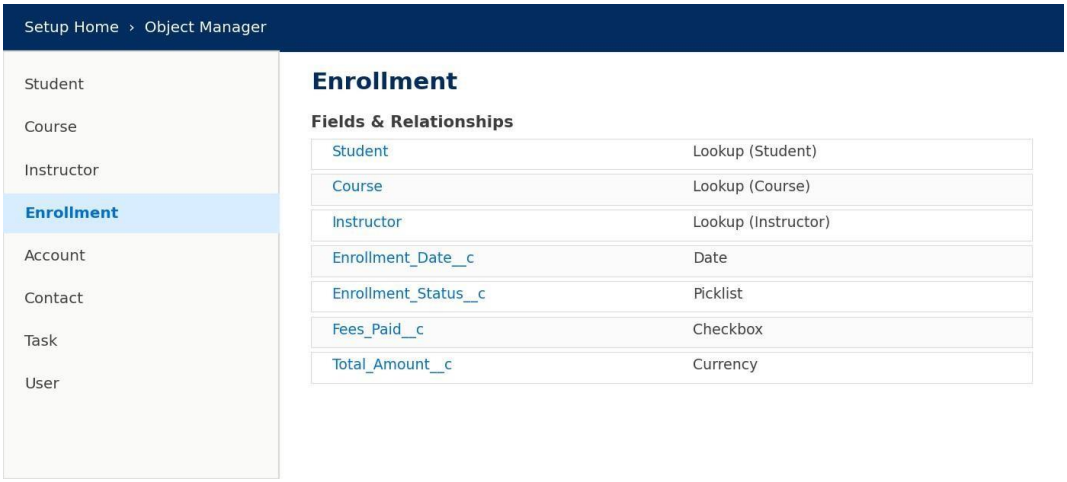


Fig. 6.5 — Object Manager view of the Enrollment object with its fields and relationships list.

Milestone 4 — Create Relationships

Activity 1: Create Lookup from Enrollment to Student

- Setup → Object Manager → Enrollment__c → Fields & Relationships → New
- Selected **Lookup Relationship** → Next
- Related To: **Student__c** → Next
- Field Label: Student (API Name: Student__c)
- Kept default Field-Level Security and Page Layout options (visible, added to layout) → Next
- Child Relationship Name: Enrollments → **Save**

Activity 2: Create Lookup from Enrollment to Course

Purpose: to link each Enrollment to a Course.

- Setup → Object Manager → Enrollment__c → Fields & Relationships → New
- Selected Lookup Relationship → Next
- Related To: Course__c → Next
- Field Label: Course (API Name: Course__c)
- Set visibility and layout options → Save

Activity 3: Create Lookup from Enrollment to Instructor

Purpose: to store which Instructor is assigned to a particular Enrollment.

- Setup → Object Manager → Enrollment__c → Fields & Relationships → New
- Chose Lookup Relationship → Next
- Related To: Instructor__c → Next
- Field Label: Instructor (API Name: Instructor__c)
- Child Relationship Name: Enrollments; marked Required (always require a value) → Save

Milestone 5 — Validation Rules

Rule 1: Student Email Domain Validation

Object: Student__c. Formula: `NOT(CONTAINS(Email__c, "@student.edu"))`. Error Message: "Email must be a valid student institutional email." Error Location: Field → Email__c.

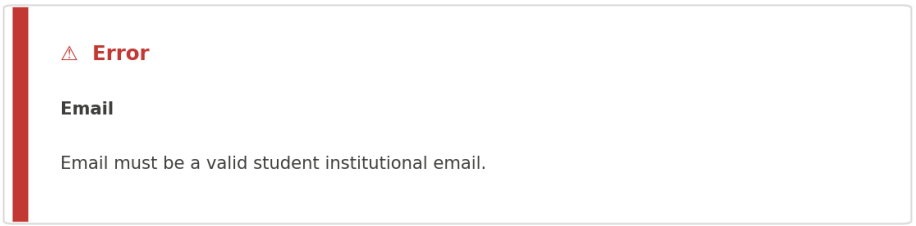


Fig. 6.1 — Validation error shown when a non-institutional email is entered on a Student record.

Rule 2: Fees Must Be Paid Before Approval

Object: Enrollment__c. Formula: `AND(ISPICKVAL(Enrollment_Status__c, "Approved"), Fees_Paid__c = FALSE)`. Error Message: "Fees must be paid before approving an enrollment." Error

Location: Top of Page.

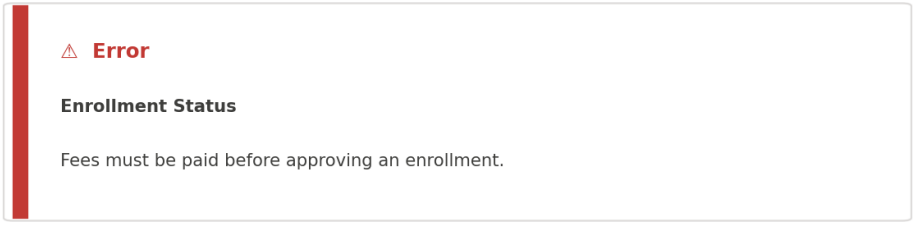


Fig. 6.2 — Validation error blocking approval of an Enrollment record with unpaid fees.

Milestone 6 — Approval Process

Activity 1: Create Approval Process for Enrollment Requests

Purpose: route Enrollment requests to the Training Manager for approval or rejection.

- Setup → Approval Processes → Manage Approval Processes For: Enrollment
- Create New Approval Process → Use Standard Setup Wizard
- Process Name: **Approval Enrollment Request**; Unique Name: Approval_Enrollment_Request
- Entry Criteria: Enrollment: Enrollment Status **equals** "Requested"
- Added Approval Step "Training Manager" with a specific Training Manager user as approver
- Final Approval Actions → Field Update: Enrollment_Status__c = "Approved"
- Final Rejection Actions → Field Update: Enrollment_Status_c = "Rejected", plus an Email Alert ("Enrollment Rejected") to the student
- Activated the approval process

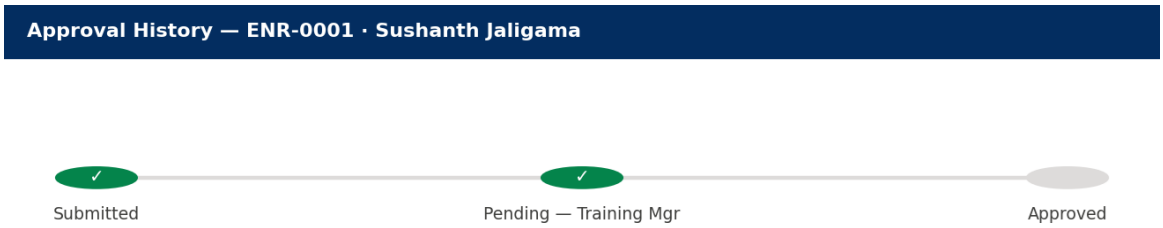


Fig. 6.3 — Enrollment pending Training Manager approval.



Fig. 6.4 — Enrollment fully approved after the Training Manager step.

Milestone 7 — Tabs

Activity 1: Creating a Tab for the Student Object

- Setup → Quick Find: "Tabs" → Custom Object Tabs → New
- Selected Student from the object list; chose a tab style/icon
- Clicked Next, assigned the tab to all profiles, then Save

Activity 2: Creating Remaining Tabs

The same steps were repeated for the Course, Instructor, and Enrollment objects, giving each object its own top-level navigation tab.

Milestone 8 — The Lightning App

Activity 1: Creating the Lightning Application for the Project

- Setup → Quick Find: "App Manager" → New Lightning App
- App Name: **Student Enrollment & Course Management** (Developer Name auto-fills)
- App Branding: selected a primary colour theme; Navigation Style: Standard Navigation
- Assigned the app to the Admin, Enrollment Officer, and Instructor profiles
- Selected Navigation Items: Student, Course, Instructor, Enrollment, Reports, Dashboards, Home, Tasks
- Reviewed settings and clicked **Save & Finish**



Fig. 6.6 — Lightning App navigation bar for Student Enrollment & Course Management, with the Enrollments tab active.

Milestone 9 — Page Layouts and Record Types

Activity 1: Create Record Types for Enrollment

Purpose: to separate New Enrollment vs Re-Enrollment processes.

- Setup → Object Manager → Enrollment__c → Record Types → New
- First record type — Existing Record Type: Master; Record Type Label: **New Enrollment**; enabled for Admin and Enrollment Officer profiles; default Page Layout selected → Save & New
- Second record type — Record Type Label: **Re-Enrollment**; enabled for the same profiles; default Page Layout selected → Save

Activity 2: Separate Page Layouts for Each Enrollment Record Type

Purpose: to show different fields for New Enrollment vs Re-Enrollment.

- Enrollment__c → Page Layouts → New, cloned from the existing Enrollment Layout, named **New Enrollment Layout** — kept Student, Course, Instructor, Enrollment Date, Status, and Fees fields relevant to first-time enrolment
- Created a second layout cloned the same way, named **Re-Enrollment Layout** — added a previous-enrollment reference, Re-Enrollment Reason, Status, and Fees fields
- Page Layout Assignment → Edit Assignment: mapped New Enrollment record type to the New Enrollment Layout and Re-Enrollment record type to the Re-Enrollment Layout, for each relevant profile → Save

Activity 3: Add Enrollment Related Lists to Parent Objects

Purpose: to see all Enrollments from Student, Course, and Instructor records.

- Student__c → Page Layouts → opened the main layout → dragged the Enrollments related list onto the layout → Save
- Course__c → Page Layouts → added the Enrollments related list → Save
- Instructor__c → Page Layouts → added the Enrollments related list → Save

Milestone 10 — Automation with Flow

Activity 1: Auto-Populate Enrollment Date (Record-Triggered Flow)

Purpose: set Enrollment_Date__c to today's date when an Enrollment is created.

- Setup → Flows → New Flow → Record-Triggered Flow → Create
- Object: Enrollment__c; Trigger the Flow When: **A record is created**
- Optimize the flow for: Actions and Related Records → Done
- Added an Update Records element — Label: "Set Enrollment Date"; used the record that triggered the flow; Field: Enrollment_Date_c → Value: $Flow.CurrentDate
- Saved as **SetEnrollmentDate** and activated

Activity 2: Send Email When Enrollment Approved

Purpose: when Enrollment_Status__c = Approved, send an email, alert the admin, and set the Student to Active.

- Created an Email Template and matching Email Alert for the student notification
- Flows → New → Record-Triggered Flow; Object: Enrollment__c, trigger on record created or updated
- Entry Condition: Enrollment_Status__c Equals "Approved"
- Added a Get Records element to fetch the related Student__c record filtered on Status Equals "Active"
- Added an Action (Email Alert) to send the "Enrollment Approved" email to the student
- Added an Action to notify the Admin where needed
- Added an Update Records element to set Student__c.Status__c = "Active"
- Saved as **Enrollment Approved Action** and activated

Activity 3: Auto-Assign Instructor Based on Course Category

Purpose: assign an Instructor automatically based on Course category, using the mapping Technical → Instructor A, Language → Instructor B, Non-Technical → Instructor C.

- Prepared Instructor__c records with codes INSTR_A, INSTR_B, INSTR_C
- Created a Record-Triggered Flow on Enrollment__c (created/updated)
- Added a Get Records element to fetch the related Course__c
- Added a Decision element branching on Course_r.Category_c: Technical / Language / Non-Technical
- For each outcome, added a Get Records element to retrieve the matching Instructor record by code, then an Update Records element to set Instructor_c = Get_Instructor_X.Id
- Saved as **AutoAssign_Instructor** and activated

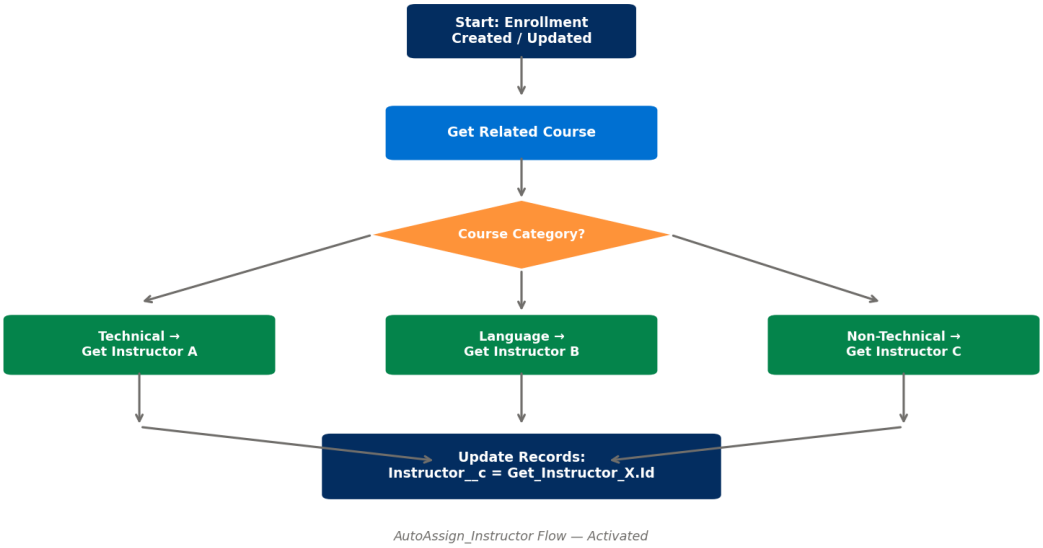


Fig. 6.7 — AutoAssign_Instructor flow: routes each Enrollment to Instructor A, B, or C by Course category.

Activity 4: Auto-Create Task for Follow-Up

Purpose: when Enrollment is "Requested", create a follow-up task for the Admin.

- Created a Record-Triggered Flow on Enrollment_c; trigger: record created or updated; condition Enrollment_Status_c = "Requested"
- Added a Get Records element to fetch the Admin user by Username / Profile
- Added a Formula resource (Date): TODAY() + 2 for the due date
- Added a Create Records element for Task — Subject: "Follow up on enrollment request"; Priority: Normal; Status: Not Started; OwnerId: Admin user Id; Related To ID / WhatId: the triggering Enrollment record; Due Date: the Today+2 formula
- Saved as **Create_Followup_Task** and activated

Milestone 11 — Reports

Activity 1: Create "Students by Status" Report

- Reports → New Report → Report Type: Students
- Group rows by Status__c
- Added columns: Name, Email, Phone
- Saved as **Students by Status**

Activity 2: Create "Enrollments by Course" Report

- New Report → Report Type: Enrollments
- Group rows by Course__r.Name
- Displayed Record Count per course
- Saved as **Enrollments by Course**

Activity 3: Create "Pending Enrollment Approvals" Report

- New Report → Enrollments report type
- Filter: Enrollment_Status__c = "Requested"
- Added columns: Student, Course, Owner, Created Date
- Saved as **Pending Enrollment Approvals**

Activity 4: Create "Revenue Report (Fees Paid)" Report

- New Report → Enrollments report type
- Filter: Fees_Paid__c = TRUE
- Grouped by Course__r.Name
- Summarized on Course_Fee__c (or equivalent fee field)
- Saved as **Revenue Report — Fees Paid**

Milestone 12 — Dashboards

Activity 1: Create "Student Management Dashboard"

- Dashboards → New Dashboard; Name: **Student Management Dashboard**
- Selected (created) a Dashboards folder → Save
- Added a **Donut Chart** component sourced from the Students by Status report
- Added a **Bar Chart** component sourced from the Enrollments by Course report
- Added a **Gauge** component sourced from the Revenue Report — Fees Paid (sum of the revenue field)
- Added a **Table** component sourced from the Pending Enrollment Approvals report
- Arranged all four components on the canvas → Save → Done

Milestone 13 — Security Setup

Activity 1: Configure Profiles

Profiles required for the application:

Profile	Access
Admin	Full access to all objects
Enrollment Officer	Create / Update Enrollments; Read/Edit Student, Course
Instructor	Read-only access to assigned Enrollments

Example for the Enrollment Officer profile: Object Settings → Enrollment_c: Read, Create, Edit; Student_c / Course_c: Read (and Edit where required) → Save. The same procedure was repeated for the Instructor profile with Read-Only access on Enrollments.

Activity 2: Create Permission Sets

- Setup → Permission Sets → New
- Created **Custom Object Access** — additional access to project objects where needed
- Created **Dashboard Editor** — Create/Customize Reports & Dashboards permission
- Assigned both permission sets to relevant users

Activity 3: Configure Sharing & Organization-Wide Defaults (OWD)

Object	OWD
Students (Student__c)	Public Read/Write
Courses (Course__c)	Public Read/Write
Enrollments (Enrollment__c)	Private

For Instructor visibility into their own assigned enrollments, a User lookup (Instructor User) was added on Enrollment, and a lightweight flow sets OwnerId to that Instructor User so each instructor sees only their own records.

7. Output Screenshots — Object Records

The following list views illustrate representative records created for each of the four core objects after data entry and configuration were completed.

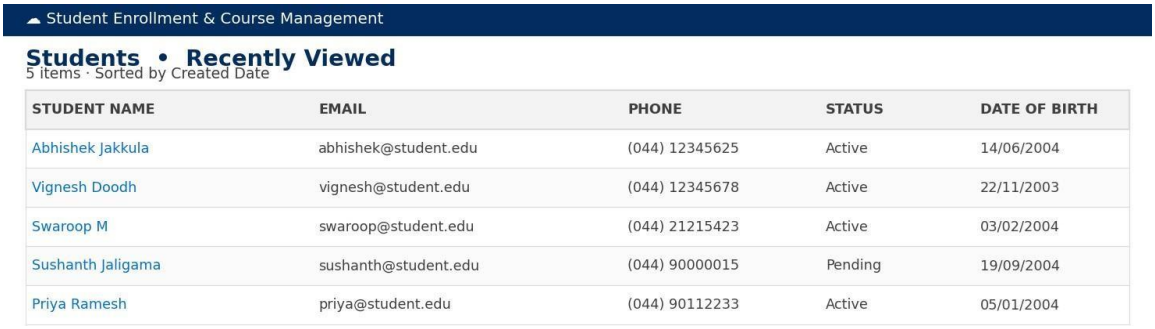


Fig. 7.1 — Student object list view showing sample student records with contact details and status.



Fig. 7.2 — Course object list view showing sample course offerings across Technical, Language, and Non-Technical categories.



Fig. 7.3 — Instructor object list view showing instructors mapped to expertise areas used by the auto-assignment flow.



Fig. 7.4 — Enrollment object list view linking students to courses with status and fee-payment tracking.

8. Output Screenshots — Reports & Dashboard

The following visuals represent the report and dashboard components configured in Milestones 11 and 12, reflecting the sample data set used during testing.

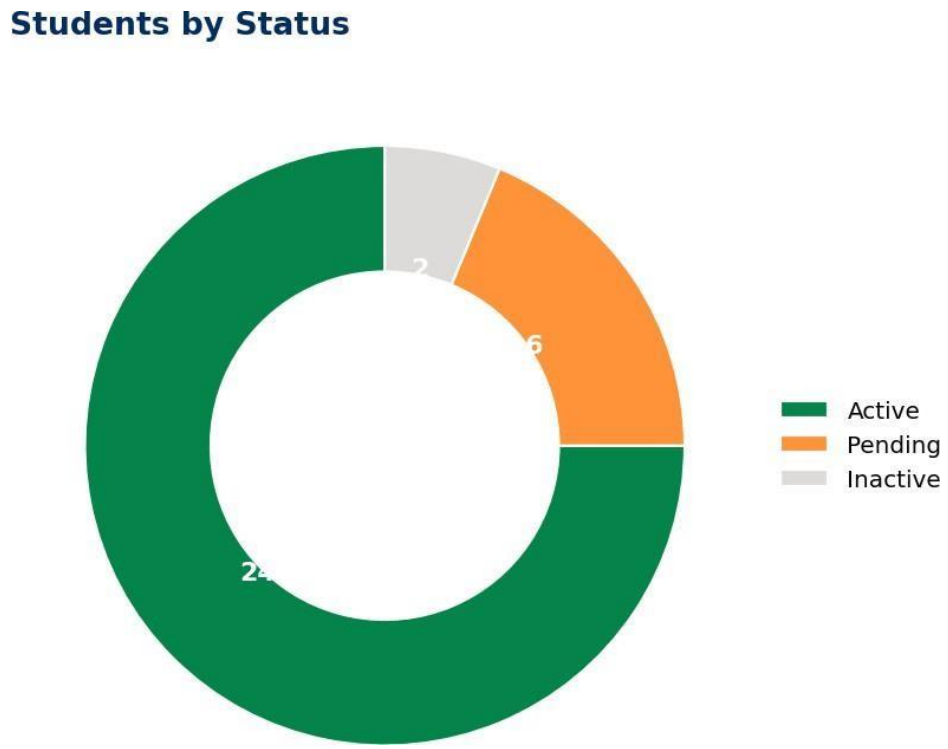


Fig. 8.1 — Students by Status donut report: 24 Active, 6 Pending, 2 Inactive.

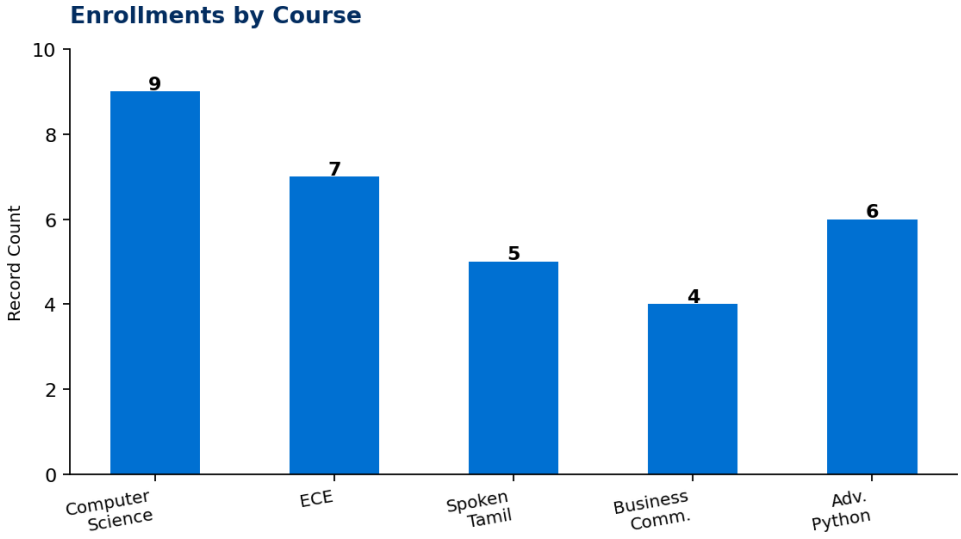


Fig. 8.2 — Enrollments by Course bar report showing demand distribution across offerings.



Fig. 8.3 — Revenue Report (Fees Paid) gauge against a ■2,50,000 target.

☁ Student Enrollment & Course Management

Pending Enrollment Approvals

2 items • Sorted by Created Date

ENROLLMENT NAME	STUDENT	COURSE	OWNER	CREATED DATE
ENR-0001	Sushanth Jaligama	Computer Science Fundamentals	Sai Kumar A	05/12/2025
ENR-0002	Vignesh Doodh	Electronic Communications & Engg.	Sai Kumar A	05/12/2025

Fig. 8.4 — Pending Enrollment Approvals table report used on the Student Management Dashboard.

9. Functional & Performance Testing

During the testing phase, the Salesforce interface was exercised to verify correct behaviour of all implemented features of SECMS. These include: student, course, instructor, and enrollment record creation; validation rule checks for mandatory fields and enrollment constraints; verification of record type behaviour for New Enrollment versus Re-Enrollment; flow execution for automated enrollment processing, instructor auto-assignment, and follow-up task creation; and approval process routing for enrollment confirmation. Generated reports and dashboards were reviewed to validate accurate data visualization, enrollment tracking, and performance monitoring, and the corresponding output screenshots are documented in Sections 7 and 8 of this report.

Test Summary

Test Area	Scenario	Expected Result	Status
Validation Rule	Student email without @student.edu doma	nSave blocked with error message	Pass
Validation Rule	Approve enrollment with Fees Paid unche	kSeadve blocked with error message	Pass
Approval Process	Submit enrollment with status Requested	Routed to Training Manager	Pass
Flow	Create new Enrollment record	Enrollment Date auto-populated	Pass
Flow	Approve enrollment	Student status set to Active; email sent	Pass
Flow	Set Course category = Technical	Instructor auto-assigned to Instructor A	Pass
Flow	Enrollment status = Requested	Follow-up task created for Admin	Pass
Security	Instructor profile login	Sees only assigned enrollments	Pass
Report	Run Students by Status	Correct grouped counts displayed	Pass
Dashboard	Refresh Student Management Dashboard	All four components render current data	Pass

10. Advantages & Disadvantages

Advantages

- Centralized data across students, courses, instructors, and enrollments
- Automated workflows for enrollment processing, notifications, and instructor assignment
- Real-time reporting and dashboard-driven monitoring
- Role-based security keeps sensitive academic data restricted appropriately
- Multi-level approval process reduces enrollment errors and revenue leakage

Disadvantages

- Salesforce platform dependency and licensing considerations at scale
- Requires initial configuration effort across objects, flows, and security
- Limited offline access for field or remote enrollment officers

11. Conclusion

The Salesforce Student Enrollment & Course Management System successfully streamlines the entire student academic lifecycle — from student registration and course creation to enrollment management, instructor assignment, approvals, revenue tracking, and reporting. By leveraging Salesforce's automation tools such as validation rules, flows, approval processes, and dashboards, the system improves data accuracy, operational efficiency, and decision-making. This project demonstrates how Salesforce can transform manual academic processes into a scalable, reliable, and user-friendly digital solution.

12. Future Scope

- Payment gateway integration for online fee collection
- Student self-service portal for enrollment and status tracking
- Analytics-driven insights using Salesforce dashboards and Einstein Analytics
- Mobile access support via the Salesforce mobile app

13. Appendix

13.1 Reference Materials

- Source Code: Salesforce Configuration (Objects, Flows, Approval Processes)
- Dataset: Salesforce Objects (Student, Course, Instructor, Enrollment)
- GitHub / Demo: Salesforce Developer Org

13.2 Glossary

ction

13.3 Document Information

Field	Detail
Report Title	Student Enrollment & Course Management System (SECMS)
Platform	Salesforce Lightning (Developer Edition)
Methodology	Agile — Sprint-based delivery across 6 sprints
Milestones Covered	14 (Developer Setup through Conclusion)
Report Sections	13

— End of Report —